

APICIA: An Automated Static Change Impact Analysis and Governance Framework for Evolving REST APIs

Abhinav Dwivedi*, Disha Dungrani*, Pragnesh Dubey*, Swapnil Bhagat†

*Department of Computer Engineering, Thakur College of Engineering and Technology, Mumbai, India

†Associate Professor, Department of Computer Engineering, Thakur College of Engineering and Technology, Mumbai, India
Email: {abhinavdwivedi, dishadungrani, pragneshdubey}@gmail.com, swapnil.bhagat@thakureducation.org

Abstract—REST APIs connect the services of modern microservice systems. When a provider team changes its API, consumer services can fail at runtime, and a textual diff of the specification does not show which changes are dangerous or who they affect. We present APICIA (API Change Impact Analyzer), a Spring Boot tool that performs change impact analysis on REST APIs using only static analysis. APICIA compares two OpenAPI specifications with 12 pluggable governance rules, scans consumer source code for Feign and RestTemplate calls to find the exact consumer files and lines affected by each breaking change, and combines both into a normalised impact score with four risk levels. It can also generate an OpenAPI 3.0 contract from Spring Boot source code with JavaParser without starting the target application, audit source code and specifications for security and performance problems (wildcard CORS, sensitive-data exposure, public endpoints, DTO reference cycles and N+1 queries), and generate client SDKs and Postman collections. On a controlled specification pair, APICIA detected all 8 injected changes (14 violations in total). In an end-to-end run with three consumer projects, it identified the two affected consumers down to file and line and raised the risk from HIGH (0.70) to CRITICAL (0.76). Static extraction recovered all 23 endpoints of APICIA’s own code base in 516 ms, and all 9 alerts raised by the security audit on that code base were confirmed as real issues. All 40 unit tests pass.

Index Terms—API evolution, breaking change detection, change impact analysis, OpenAPI, static analysis, blast radius, microservices.

I. INTRODUCTION

Modern software systems are built from services that communicate through REST APIs. Because these services are deployed independently, a provider team can change its API without noticing that other teams depend on the changed part. Removing an endpoint, changing a parameter type or adding a mandatory field causes consumer services to fail, often only after the change has reached production.

Breaking changes are common. Xavier et al. analysed 317 Java libraries and found that 14.78% of API changes break backward compatibility, while 2.54% of client projects were actually impacted [5]. For web APIs the situation is worse, because clients often cannot remain on an old version once the provider stops serving it [2]. In a study of 41,627 OpenAPI descriptions, only 0.6% of APIs deprecated operations before removing them, whereas 24.6% removed operations directly [12]. In interviews with practitioners, the most fre-

quently reported challenge of microservice API evolution was that manual change impact analysis is error-prone [15].

A. Shortcomings of Existing Approaches

- 1) **No ranking:** textual and specification diffs list what changed but do not say which changes are dangerous.
- 2) **No consumer awareness:** most diff tools analyse a specification in isolation. A breaking change on an endpoint with no consumers is reported with the same urgency as one on an endpoint used by many services. Consumer-driven contract testing solves this, but only for consumers that write and maintain contract tests [8].
- 3) **Dependence on a running application:** reflection-based OpenAPI generators need the target application context to start, which requires its databases and configuration.
- 4) **Security and performance checks are separate:** problems such as wildcard CORS or sensitive fields in responses are handled by separate tools, not as part of change review.

B. Contributions

- A static extraction engine that generates OpenAPI 3.0 contracts from Spring Boot controllers using JavaParser, without starting the target application.
- A rule engine with 12 pluggable governance rules and a structural score D_{struct} with a severity floor, so that a single breaking change is not diluted by the size of the API.
- A consumer blast radius module that links every breaking change to the consumer projects, files and lines that call the changed endpoint, giving D_{blast} .
- A weighted impact score S_{total} with four risk levels, a static security and performance audit, and SDK and Postman generation.
- An evaluation on a controlled specification pair, synthetic consumer projects and APICIA’s own code base, with all results measured from the implementation.

II. RELATED WORK

A. API Evolution and Breaking Changes

Dig and Johnson showed that most API-breaking changes in Java frameworks are refactorings such as renames and

moves [1]. Xavier et al. measured the frequency and client impact of breaking changes at large scale [5], and Brito et al. studied why developers break APIs [6]. Ochoa et al. examined how releases on Maven Central follow semantic versioning and how breaking changes affect clients [10], and Jayasuriya et al. and Venturini et al. studied how breaking changes manifest in client projects [13], [14]. Lamothe et al. survey the API evolution literature [11].

B. Evolution of Web APIs

Li et al. identified 16 change patterns in web APIs [2]. Wang et al. studied how client developers react to REST API evolution [3], and Sohan et al. examined how web API changes are communicated [4]. Di Lauro et al. showed that operations are usually removed without deprecation [12], and Lercher et al. reported that practitioners find manual change impact analysis error-prone [15].

C. Breaking Change Detection Tools

APIDiff detects breaking and non-breaking changes between versions of Java libraries [7]. Lercher et al. generate OpenAPI descriptions from Java source code [16], and AutoGuard generates OpenAPI descriptions of two versions of a Spring Boot service from source code and reports breaking changes in CI pipelines [17]. AutoGuard is the closest work to APICIA. APICIA differs in that it produces a risk score, maps each breaking change to the consumer code that will break, audits security and performance, and generates client SDKs.

D. Contract Testing and Continuous Evolution

Consumer-driven contract testing detects incompatibilities reliably but needs every consumer to maintain tests [8]. Knoche and Hasselbring discuss continuous API evolution in enterprise systems with many coexisting consumers [9]. APICIA instead discovers consumers from their source code, so no work is required from consumer teams.

III. SYSTEM ARCHITECTURE

APICIA is a Spring Boot application with a layered architecture (Fig. 1). Six REST controllers expose 23 endpoints; the service layer contains five modules:

- 1) **Static extraction:** parses Spring Boot controllers with JavaParser and emits an OpenAPI 3.0 document.
- 2) **Static Governance Module (SGM):** compares two OpenAPI specifications with 12 rules and computes D_{struct} .
- 3) **Dependency scanner and blast radius:** scans consumer projects and links breaking changes to consumer code, giving D_{blast} .
- 4) **Impact scoring:** combines D_{struct} and D_{blast} into S_{total} and a risk level.
- 5) **Security and performance scanner (SAM) and SDK generator:** audits code and specifications, and generates SDKs and Postman collections.

Specifications, analysis reports, violations, consumer projects and their dependencies are stored in PostgreSQL through Spring Data JPA.

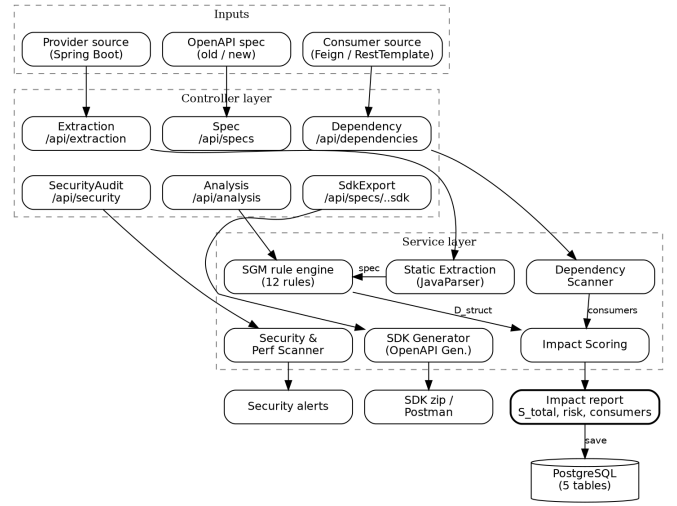


Fig. 1. Architecture of APICIA.

Algorithm 1 Static endpoint extraction

Require: source root D

Ensure: endpoint list E , OpenAPI document S

```

1:  $E \leftarrow \emptyset$ 
2: for all compilation unit  $u$  parsed from  $D$  do
3:   for all class  $c \in u$  annotated @RestController or
     @Controller do
4:      $p_{base} \leftarrow$  class-level @RequestMapping path
5:     for all method  $m \in c$  with a mapping annotation do
6:       read HTTP method and sub-path from the annotation
7:       read @PathVariable, @RequestParam,
         @RequestBody
8:        $E \leftarrow E \cup \{(p_{base} + p_{sub}, method, params, body)\}$ 
9:     end for
10:  end for
11: end for
12: apply security configuration to each endpoint
13:  $S \leftarrow \text{BUILDOPENAPI}(E)$ 
14: return  $E, S$ 

```

IV. STATIC ENDPOINT EXTRACTION

APICIA parses every Java file under a source root with JavaParser. It does not need to compile or start the target application; the analyser itself runs on a normal JVM. Algorithm 1 summarises the process.

Java types are mapped to OpenAPI types: int, short and byte to integer/int32; long to integer/int64; float, double and BigDecimal to number; String, char and UUID to string; date and time types to string with format date-time; and List, Set, Collection and Iterable to arrays. Bean Validation annotations on parameters are mapped to schema constraints: @NotNull, @NotBlank and @NotEmpty to required; @Size to minLength/maxLength; @Min and @Max to minimum/maximum; and @Pattern to pattern. Frame-

TABLE I
COMPARISON OF APICIA WITH RELATED APPROACHES

Capability	Spec diff	Contract testing	APIDiff [7]	AutoGuard [17]	APICIA
Detects REST breaking changes	Partly	Yes	No (Java libraries)	Yes	Yes (12 rules)
Contract from source code	No	No	—	Yes	Yes
Risk score	No	No	No	No	Yes (S_{total})
Affected consumers (file, line)	No	Tested ones only	No	No	Yes
Effort for consumer teams	None	High	None	None	None
Security / performance audit	No	No	No	No	Yes
SDK / Postman generation	No	No	No	No	Yes

work parameter types such as `HttpServletRequest` are excluded. The generated document records the controller class and method of each operation as vendor extensions, which later lets a violation point to the exact code responsible.

V. STATIC GOVERNANCE MODULE

Each rule implements a common `DesignRule` interface and is a Spring component; the rule engine receives the list of all rules through dependency injection, so a new rule is added by writing a single class. Each rule runs inside its own exception handler, so a failure in one rule does not stop the others. Table II lists the 12 implemented rules. The identifier SGM-008 is unused.

A. Structural Score D_{struct}

Let b , w and i be the numbers of breaking (including critical), warning and info violations, and E the number of operations in V_{new} . With configured weights $w_B = 0.75$, $w_W = 0.35$ and $w_I = 0.05$, the weighted density is

$$r = \frac{w_B b + w_W w + w_I i}{E} \quad (1)$$

and the structural score applies a severity floor:

$$D_{struct} = \begin{cases} \min(1, \max(w_B, r)) & b > 0 \\ \min(1, \max(w_W, r)) & b = 0, w > 0 \\ \min(1, r) & \text{otherwise} \end{cases} \quad (2)$$

with $D_{struct} = 0$ when $E = 0$. Without the floor, a single removed endpoint in an API of 60 operations would give $r \approx 0.0125$ and would look harmless, although every client of that endpoint now receives HTTP 404. With the floor, any breaking change yields $D_{struct} \geq 0.75$.

VI. CONSUMER BLAST RADIUS

Consumer projects are registered by name and source path. The dependency scanner parses their source with `JavaParser` and records two kinds of calls: methods of `@FeignClient` interfaces, whose mapping annotations give the path, and `RestTemplate` calls such as `getForObject`, whose first argument gives the URL. Each call is stored with its file, line, HTTP method and a normalised path in which the host is removed and path variables and numeric segments are replaced by a placeholder, so that `http://host/api/v1/users/42` matches `/api/v1/users/{id}`.

Algorithm 2 Blast radius

Require: violations V , dependency store R

Ensure: impacted endpoints with consumers, D_{blast}

```

1:  $C \leftarrow \emptyset$ 
2: for all  $v \in V$  with severity BREAKING or CRITICAL
   do
3:    $deps \leftarrow R.FIND(method(v), normalise(path(v)))$ 
4:   for all  $d \in deps$  do
5:      $C \leftarrow C \cup \{d.project\}$ ; record  $(v, d.file, d.line)$ 
6:   end for
7: end for
8: return impacted endpoints,  $\min(1, |C|/10)$ 
```

For every violation with severity BREAKING or CRITICAL, APICIA looks up stored dependencies with the same HTTP method and normalised path and collects the distinct consumer projects C (Algorithm 2). With saturation constant $M = 10$,

$$D_{blast} = \min\left(1, \frac{|C|}{M}\right) \quad (3)$$

VII. IMPACT SCORE AND RISK LEVELS

The total impact score is

$$S_{total} = \min(1, 0.70 D_{struct} + 0.30 D_{blast}) \quad (4)$$

and the risk level is LOW for $S_{total} < 0.20$, MEDIUM for $0.20 \leq S_{total} < 0.50$, HIGH for $0.50 \leq S_{total} < 0.75$ and CRITICAL for $S_{total} \geq 0.75$. All weights and thresholds are configurable. The structural term is weighted higher because the consumer count depends on which projects have been registered and is therefore a lower bound. A consequence of $w_1 = 0.70$ is that structure alone cannot exceed 0.70, which lies in the HIGH band: the CRITICAL level is reached only when registered consumers are actually affected.

VIII. SECURITY AND PERFORMANCE AUDIT

The scanner works on Java source code or on an OpenAPI specification, and every alert includes a location and a suggested fix. It implements five checks:

- **SEC-CORS-001:** `@CrossOrigin` with origin `*`, or with no origins (which defaults to `*`).
- **SEC-LEAK-001/002:** DTO fields or response properties whose names match sensitive patterns (e.g. password,

TABLE II
GOVERNANCE RULES IMPLEMENTED IN APICIA

Rule	Condition	Severity
SGM-001	Path does not match $\wedge/v[0-9]+/$ (no version prefix)	BREAKING
SGM-002	A path segment contains an underscore (snake_case)	WARNING
SGM-003	An operation present in V_{old} is absent in V_{new}	BREAKING
SGM-004	A parameter's type changes	BREAKING
SGM-005	A request field becomes required / a required field becomes optional	BREAKING / WARNING
SGM-006	The HTTP method of an existing path changes	CRITICAL
SGM-007	Authentication removed or made public; added; or its type changed	CRITICAL / BREAKING / WARNING
SGM-009	A path is renamed: removed and added operations share controller class, controller method and HTTP method	BREAKING
SGM-010	A path segment has uppercase letters or uses a singular resource noun (e.g. /user)	WARNING
SGM-011	A path contains an action verb (e.g. /createUser)	WARNING
SGM-012	An operation has no documented error responses, or an unstructured error body	WARNING
SGM-013	A success response is removed or its schema changes	BREAKING

token, secret, cvv, api key), unless the field is annotated @JsonIgnore; and sensitive values passed as URL parameters.

- **SEC-NOAUTH-001:** endpoints that require no authentication; public PUT, DELETE and PATCH endpoints are reported as unauthenticated state modification.
- **PERF-CYCLE-001:** cycles in the DTO reference graph, found by depth-first search after removing fields annotated @JsonIgnore or @JsonBackReference.
- **PERF-NPLUS1-001:** repository calls inside for, for-each or while loops.

The comparison endpoint also runs the specification audit on V_{new} and returns its result with the impact report.

IX. SDK AND POSTMAN GENERATION

APICIA generates client SDKs on request by running the OpenAPI Generator library (version 7.4.0) on a stored or supplied specification; the output is zipped and returned. The default target is typescript-axios, and any generator supported by OpenAPI Generator, such as python or java, can be requested. A Postman Collection v2.1.0 export is implemented directly: it walks every operation, groups requests into folders, fills example bodies from the schema and adds a baseUrl variable.

X. IMPLEMENTATION

APICIA is implemented in Java 17 with Spring Boot 4.1.0, Spring Data JPA and PostgreSQL. It uses swagger-parser 2.1.22 for OpenAPI parsing, JavaParser 3.26.3 for source analysis and OpenAPI Generator 7.4.0 for SDKs. The code base has 71 main source files, 5 JPA entities and 6 controllers (/api/specs, /api/analysis, /api/extraction, /api/dependencies, /api/security and the SDK export endpoints). The test suite has 40 JUnit 5 tests in 11 classes, using Mockito for repositories.

XI. EVALUATION

All results below were measured by running the implementation with the default configuration on OpenJDK 17; timings are taken after one warm-up run.

A. Setup

- **Specification pair:** openapi_v1.json (5 operations) and openapi_v2.json (6 operations) from the repository. Version 2 contains 8 known changes: a parameter type change, a new required field, two removed GET operations, a GET changed to POST, a removed security requirement, an unversioned path and a snake_case path. It also returns a passwordHash field.
- **Consumer projects:** three small projects written for this test: *mobile-app* (a Feign client calling GET /v1/posts), *web-bff* (RestTemplate calls to GET /v1/posts and GET /v1/old-endpoint) and *analytics* (a call to GET /v1/stats, which does not change). The real dependency scanner parsed them and its output was passed to the real comparison service, with repositories held in memory.
- **Own code base:** APICIA's 71 main source files, used for extraction and security audit.

B. Change Detection

The 12 rules reported 14 violations: 6 breaking (one of them CRITICAL) and 8 warnings (Table III, Fig. 2). Each of the 8 injected changes was detected by the rule designed for it. The remaining 6 were SGM-012 warnings, correctly reporting that six operations document no error responses. No violation was reported that did not correspond to a real property of the specifications.

C. Impact Score and Blast Radius

The weighted density was $(6 \times 0.75 + 8 \times 0.35)/6 = 1.22$, so D_{struct} was clamped to 1.0 and, with no consumers, $S_{total} = 0.70$ (HIGH). The dependency scanner found four calls in the three consumer projects. APICIA reported two affected consumers (Fig. 3): GET /v1/posts is called by *mobile-app* (PostClient.java, line 6) and *web-bff* (Svc.java, line 5), and GET /v1/old-endpoint by *web-bff* (line 6). *web-bff* was counted once. *analytics* was correctly excluded, and the breaking changes on /v1/users were correctly linked to no consumer because none calls it.

TABLE III
VIOLATIONS ON THE SPECIFICATION PAIR

Rule	Severity	Endpoint	Change
SGM-001	BREAKING	/users	no version prefix
SGM-002	WARNING	/v1/user_profile	snake_case
SGM-003	BREAKING	/v1/old-endpoint	GET removed
SGM-003	BREAKING	/v1/posts	GET removed
SGM-004	BREAKING	/v1/users	id: int→string
SGM-005	BREAKING	/v1/users	new required age
SGM-006	CRITICAL	/v1/posts	GET→POST
SGM-007	WARNING	/v1/secure	security removed
SGM-012	WARNING	6 operations	no error resp.

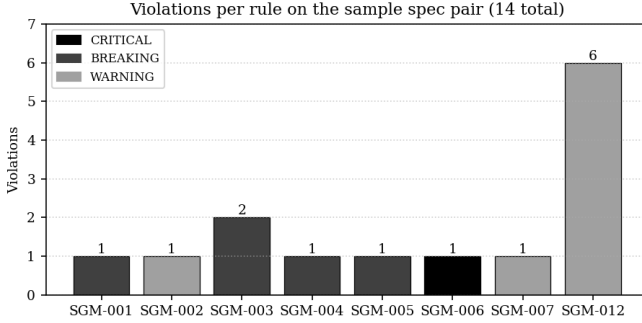


Fig. 2. Violations per rule on the specification pair.

With $|C| = 2$, $D_{blast} = 0.2$ and $S_{total} = 0.76$, which raised the risk level from HIGH to CRITICAL. Fig. 4 shows how S_{total} varies with the number of affected consumers for $D_{struct} = 1.0$; only the two-consumer point comes from an actual scan.

D. Extraction, Audit and SDK Generation

Static extraction on APICIA's own code recovered all 23 REST endpoints of its 6 controllers (15 GET, 8 POST) in 516ms. The security audit of the same code raised 9 alerts, all rated HIGH, and each was checked by hand: the 6 SEC-CORS-001 alerts correspond exactly to the 6 controllers annotated `@CrossOrigin(origins = "*")`, and the 3 PERF-NPLUS1-001 alerts are real repository calls inside loops, one of them in the blast radius computation itself. The audit of `openapi_v2.json` raised one CRITICAL alert for the `passwordHash` response field and listed 6 endpoints without security requirements. Table IV lists timings and SDK sizes.

XII. DISCUSSION AND LIMITATIONS

Evaluation scope. The specification pair is small and was written by the authors, and the consumer projects are synthetic. The results show that the rules behave as designed; they do not establish accuracy on real API histories, and changes outside the rule set cannot be measured with this data.

Score saturation. On the sample, six SGM-012 warnings alone contributed 2.1 to the numerator of r , and D_{struct} saturated at 1.0. On small APIs with many warnings, a worse

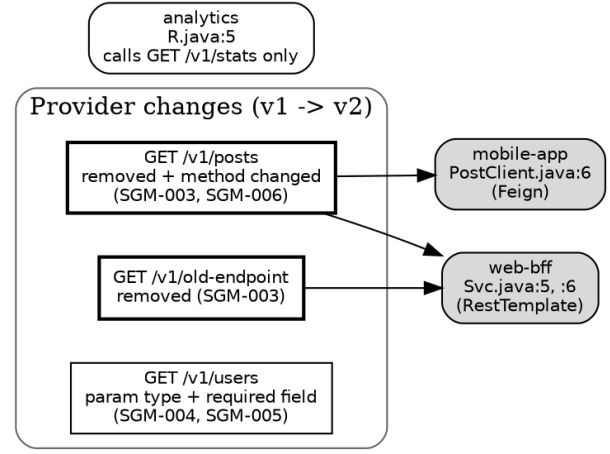


Fig. 3. Measured blast radius: breaking changes linked to consumer code.

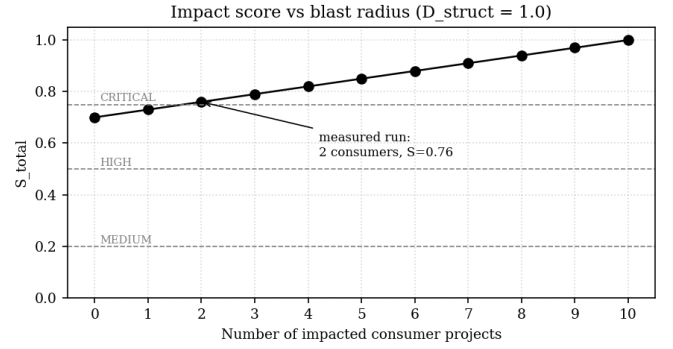


Fig. 4. S_{total} against the number of affected consumers ($D_{struct} = 1.0$).

change can no longer be distinguished. The floor protects the lower end of the scale, but the upper end saturates.

Structural analysis only. Changes that keep types but alter meaning, such as a change of currency unit, cannot be detected [13].

Consumer coverage. Only Feign and RestTemplate calls are detected; WebClient, other HTTP libraries and runtime-built URLs are missed, so $|C|$ is a lower bound.

Other limitations. Exceptions inside rules are ignored silently; the security score computed by the audit is not yet part of S_{total} ; weights and thresholds are chosen by design reasoning rather than calibrated; and extraction and scanning support only Java with Spring Boot. The self-audit also found real issues in APICIA itself, which will be fixed.

XIII. CONCLUSION AND FUTURE WORK

APICIA answers two questions about a REST API change using static analysis only: is it breaking, and which consumer code will break. It combines 12 governance rules, a structural score with a severity floor and a consumer blast radius into one explainable impact score, and adds contract extraction from source, a security and performance audit, and SDK generation. On the evaluated data it detected all injected

TABLE IV
MEASURED PERFORMANCE

Operation	Result
12 rules on specification pair	3.1 ms
Full comparison incl. blast radius (first call)	563 ms
Static extraction (71 files, 23 endpoints)	516 ms
Security audit of source (71 files)	2,299 ms
SDK: typescript-axios (11 files)	165 ms
SDK: python (34 files)	195 ms
SDK: java (46 files)	233 ms
Postman export (6 requests)	7 ms
Unit tests	40/40 pass

changes, identified the affected consumers down to file and line, and changed the risk decision from HIGH to CRITICAL once real consumers were found.

Future work includes evaluation on real open-source API histories, integration as a Git pre-push hook and pull-request bot, adding the security score to S_{total} , reducing score saturation and calibrating weights on real data, supporting more client libraries in the dependency scanner, and extending extraction beyond Spring Boot.

REFERENCES

- [1] D. Dig and R. Johnson, “How do APIs evolve? A story of refactoring,” *J. Softw. Maint. Evol.: Res. Pract.*, vol. 18, no. 2, pp. 83–107, 2006.
- [2] J. Li, Y. Xiong, X. Liu, and L. Zhang, “How does web service API evolution affect clients?” in *Proc. IEEE ICWS*, 2013, pp. 300–307.
- [3] S. Wang, I. Keivanloo, and Y. Zou, “How do developers react to RESTful API evolution?” in *Proc. ICSOC*, Springer, 2014, pp. 245–259.
- [4] S. M. Sohan, C. Anslow, and F. Maurer, “A case study of web API evolution,” in *Proc. IEEE World Congr. Services (SERVICES)*, 2015, pp. 245–252.
- [5] L. Xavier, A. Brito, A. Hora, and M. T. Valente, “Historical and impact analysis of API breaking changes: A large-scale study,” in *Proc. IEEE SANER*, 2017, pp. 138–147.
- [6] A. Brito, L. Xavier, A. Hora, and M. T. Valente, “Why and how Java developers break APIs,” in *Proc. IEEE SANER*, 2018, pp. 255–265.
- [7] A. Brito, L. Xavier, A. Hora, and M. T. Valente, “APIDiff: Detecting API breaking changes,” in *Proc. IEEE SANER*, 2018, pp. 507–511.
- [8] J. Lehvä, N. Mäkitalo, and T. Mikkonen, “Consumer-driven contract tests for microservices: A case study,” in *Proc. PROFES*, LNCS 11915, Springer, 2019, pp. 497–512.
- [9] H. Knoche and W. Hasselbring, “Continuous API evolution in heterogeneous enterprise software systems,” in *Proc. IEEE ICSE*, 2021, pp. 58–68.
- [10] L. Ochoa *et al.*, “Breaking bad? Semantic versioning and impact of breaking changes in Maven Central,” *Empirical Softw. Eng.*, 2022, doi: 10.1007/s10664-021-10052-y.
- [11] M. Lamothe, Y.-G. Guéhéneuc, and W. Shang, “A systematic review of API evolution literature,” *ACM Comput. Surv.*, vol. 54, no. 8, pp. 171:1–171:36, 2022.
- [12] F. Di Lauro, S. Serbout, and C. Pautasso, “To deprecate or to simply drop operations? An empirical study on the evolution of a large OpenAPI collection,” in *Proc. ECSA*, LNCS, Springer, 2022.
- [13] D. Jayasuriya, V. Terragni, J. Dietrich, S. Ou, and K. Blincoe, “Understanding breaking changes in the wild,” in *Proc. ACM ISSTA*, 2023, pp. 1433–1444.
- [14] D. Venturini, F. R. Côgo, I. Polato, M. A. Gerosa, and I. S. Wiese, “I depended on you and you broke me: An empirical study of manifesting breaking changes in client packages,” *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 4, pp. 94:1–94:26, 2023.
- [15] A. Lercher, J. Glock, C. Macho, and M. Pinzger, “Microservice API evolution in practice: A study on strategies and challenges,” *J. Syst. Softw.*, art. 112110, 2024.
- [16] A. Lercher, C. Macho, C. Bauer, and M. Pinzger, “Generating accurate OpenAPI descriptions from Java source code,” arXiv:2410.23873, 2024.
- [17] A. Lercher, C. Bauer, C. Macho, and M. Pinzger, “AutoGuard: Reporting breaking changes of REST APIs from Java Spring Boot source code,” in *Proc. IEEE SANER*, 2025, pp. 814–818.